

大数据学期项目计划

在过去的14周里，我们一直在孤立地构建各个组件。你们编写了Spark任务，启动了Kafka代理，查询了文档存储，并将文本嵌入到向量空间中。

在现实世界中——无论你是加入大型科技公司的基础设施团队，还是A轮创业公司——没有人会为那些“在我的笔记本上能运行”的孤立脚本付费。行业为可靠、自动化、可扩展的**系统**付费。

在你们的毕业设计项目中，你们将构建这样一个系统。

以下是课程最后阶段的参与规则、可交付成果和评分机制。请仔细阅读。将这份文件视为你们的第一份工程需求规格说明书。

1. 任务：选择你的技术方向

正如课堂上所讨论的，你们将组成团队，并从三个架构方向中选择一个。虽然业务领域不同，但期望达到的技术严谨性是相同的。

- **方向 A：** 实时欺诈与零售引擎（流处理、Kafka、低延迟）
- **方向 B：** 金融科技哨兵（数据治理、湖仓一体分层架构、Delta Lake）
- **方向 C：** 企业人工智能记忆 / 检索增强生成流水线（非结构化文本、向量数据库、大型语言模型）

你们的目标： 设计架构、摄取数据、处理数据并提供服务。

2. 可交付成果

你们的团队将主要根据两个主要可交付成果进行评估。没有传统的“期末考试”——这个项目就是你们能力的证明。

A. 系统与报告论文（工程设计文档）

你们将提交一份正式的技术报告。不要像写学术论文那样写；要像写科技公司的**设计文档**那样写。它必须包括：

- **执行摘要：** 这个数据流水线解决什么业务问题？
- **架构图：** 清晰的数据流视觉图（数据摄取 -> 处理 -> 存储 -> 服务）。
- **技术权衡：** 为什么选择Spark而不是Pandas？为什么选择Mongo而不是Postgres？为你的选择辩护。
- **云财务与成本估算：** 如果将此系统部署在阿里云/华为云上以处理每天10TB的数据，大概需要多少成本？（计算、存储、API调用）。
- **事后剖析：** 什么出了问题？已知的bug有哪些？（在此诚实可得高分；隐藏bug会扣分）。
- **代码库：** 一个Jupyter Notebook，准确解释如何运行你们的流水线。

B. 最终演示（“技术总监汇报”）

在最后一周，你们的团队将向全班进行演示。

- **形式：** 最多10分钟，随后5分钟问答。
- **演示：** 向我们展示系统运行。（专业提示：最好提前录制好你们工作流水线的视频。现场演示依赖于“演示之神”，而众所周知他们非常残酷）。

- **答辩：** 我将扮演你们的技术总监。我会问你们，如果一个节点发生故障、数据延迟到达、或者API限流，会发生什么。准备好为你们的架构辩护。

3. 评分机制（团队 vs. 个人）

构建数据系统是一项团队运动。在我自己的公司里，一个拒绝协作、拖累团队士气的杰出工程师，对组织而言是净负值。因此，你们的分数将同时反映系统质量和你们的专业贡献。

第一步：团队基础分（满分100分）

我将根据报告（60%）和演示/展示（40%）来评估整个项目。假设你们的团队构建了一个很棒的系统，得到了 **90/100** 分。

第二步：个人乘数（同行评议）

项目结束时，每个学生将提交一份保密的同行评议表，按照评分标准（例如，编码、沟通、可靠性）评价他们的队友。

- 我将使用这些评价来计算一个**个人乘数（范围从0.5到1.15）**。
- **计算公式：** 个人分数 = 团队基础分 × 个人乘数

场景举例：

- **可靠成员（乘数 = 1.0）：** 你恰好完成了自己应承担的那份工作。你的分数是 $90 \times 1.0 = 90$ 。
- **幽灵成员（乘数 = 0.7）：** 你缺席会议，零代码提交，让你的团队陷入恐慌。你的分数是 $90 \times 0.7 = 63$ 。
- **技术带头人（乘数 = 1.1）：** 你超越期望，调试队友的代码并协调最终的代码合并。你的分数是 $90 \times 1.1 = 99$ 。

*注意：如果不属实，你 cannot 通过给每个人打满分来钻空子。如果不能说明每段代码是谁写的，我会将小组中每个人的个人乘数都记为0.6。

4. 生存指南：项目开始前与进行中

开始之前：

- **组建平衡的团队。** 不要把四个机器学习建模人员放在一个团队里；你们需要有人愿意编写那些不引人注目但必要的数据管道工作（Docker、Airflow、Spark）。
- **积极控制范围。** 一个能完美端到端运行的简单系统就是**A**。一个庞大、复杂但在第二步就崩溃的系统是**C**。

冲刺期间：

- **警惕"集成地狱"。** 不要等到第16周才把你的Python爬虫连接到你的Kafka主题，再连接到你的Spark任务。首先构建一个"行走的骨架"——在第三天就让一行模拟数据通过整个流水线。然后，再让它变得复杂。
- **人工智能辅助工具的使用：** 鼓励你们使用大型语言模型来生成样板代码或调试语法错误——这就是现代工程的工作方式。然而，**你们对代码负有严格责任**。如果我在你们演示时指着一块代码，问它如何处理内存分区，而你们的回答是"这是大模型写的，我不知道"，你们的答辩将不及格。

结语

这个项目旨在成为你们作品集的核心部分。当你们在顶级科技公司面试，被问到“请谈谈你处理复杂数据的一次经历”时，这就是你们将要讲述的故事。

拥抱挑战。构建一个你们引以为豪的作品。

如果遇到架构上的障碍，我的办公时间随时开放。让我们开始工作吧。

附录 A：毕业设计方向规范

为确保评分的公平性，每个团队必须构建一个包含**数据摄取、处理、存储和服务**的系统。然而，该系统的形态取决于你们选择的方向。

请仔细阅读以下规范。如果你们的架构不满足所选方向的“必需架构”约束，你们的系统将被视为不完整。

方向 A：实时欺诈与风险监控平台

核心挑战：毫秒级决策与不可变历史存储相结合。

场景：

你们正在为一家金融科技独角兽公司或一个大型电子商务平台构建风险引擎。全球范围内每分钟发生数百万笔交易。你们必须在交易发生时检测欺诈行为（流处理），通知运营团队，并为每笔交易存储一个永久的、符合法律规定的记录（批处理/湖仓一体），供数据科学团队训练未来模型使用。

必需架构（基本框架）：

- 数据摄取（事件流）：**你们必须编写一个Python代码，持续生成模拟的JSON格式交易数据（用户ID、金额、时间戳、地点、商户等），并将其推送到**Kafka**主题中。
- 处理（流引擎）：**使用**Spark结构化流**消费Kafka主题。你们必须至少实现一个有状态操作（例如，一个滚动/滑动窗口，计算“过去5分钟内每个用户的交易次数”，以检测速度欺诈）。
- 存储（湖仓一体）：**
 - 将所有原始事件写入**原始层Delta表**。
 - 将掩码版本（对用户的个人身份信息/电子邮件进行加密哈希处理）写入**清洗层Delta表**。
- 服务（告警接收端）：**当Spark根据你们的规则检测到欺诈事件时，它必须将该特定事件写入一个操作型数据库（例如，**MongoDB** 或 **PostgreSQL**）或一个实时控制台/仪表板，供欺诈团队审查。

你们的“空白画布”（创意自由）：

- 领域：**这是信用卡欺诈？加密货币洗售？电子商务账户被盗？你们决定。
- 逻辑：**你们定义欺诈启发式规则。一个用户的IP在10分钟内从纽约跳到东京是否会触发警报？凌晨3:00的一笔5,000美元购买是否会触发警报？

什么能得'A'：我可以运行你们的Python Kafka代码，有意注入一笔“坏”交易，并在几秒钟内看到它出现在你们的告警数据库中，同时验证原始数据安全地落入了你们的Delta Lake。

方向 B：智能客户支持与检索增强生成助手

核心挑战：非结构化数据流水线、语义搜索和大型语言模型接地。

场景：

你们的企业拥有一个庞大的内部存储库，里面充满了混乱的非结构化数据——过去的客户支持工单、PDF手册和Slack日志。员工们花费数小时寻找答案。你们的任务是为一个检索增强生成系统构建数据流水线。你们不仅仅是在构建一个ChatGPT包装器；你们是在构建一个稳健的数据工程后端，为人工智能提供数据。

必需架构（基本框架）：

- 数据摄取（批处理/文档）：** 摄取一个大型文本数据集（例如，Kaggle上的亚马逊评论数据集、ArXiv研究论文或软件文档）。
- 处理（清洗与分块）：** 使用PySpark或Python清洗文本（移除HTML标签，处理错误编码）。关键的是，你们必须编写逻辑将文本分割成语义上有意义的“块”，并提取元数据（作者、日期、类别）。
- 嵌入（向量化）：** 将你们的文本块通过嵌入模型（例如，HuggingFace的 `all-MiniLM-L6-v2` 或OpenAI API）进行处理，将文本转换为密集向量。
- 存储与服务（向量数据库）：** 将向量和元数据加载到向量数据库（例如，**ChromaDB**、**Milvus** 或 **Qdrant**）。
- 查询应用程序：** 编写一个Python脚本，用户输入自然语言问题。系统必须执行**混合搜索**（语义向量搜索 + SQL风格的元数据过滤，例如，“查找关于‘登录错误’的文档，但仅限2023年的”）。将检索到的上下文传递给大型语言模型以生成最终答案。

你们的“空白画布”（创意自由）：

- 数据集：** 选择一个你感兴趣的领域。医学期刊？法律合同？电子游戏传说？
- 分块与搜索策略：** 你们如何分块数据以及如何调整检索参数（前K个结果、相似度阈值）完全由你们决定。

什么能得'A'： 你们能够展示一个干净、自动化的流水线，它接受一个包含杂乱文本文件的原始文件夹，并将其转变为一个可搜索的向量索引。当被查询时，你们的系统能够检索到确切正确的段落，并且大型语言模型能准确引用它。

方向 C：城市交通与智慧城市数据平台

核心挑战： 湖仓一体分层架构、复杂连接和运营分析。

场景：

你们是城市交通部门（或像滴滴这样的公司）的首席数据工程师。你们拥有大量历史数据（站点位置、历史出行模式）以及连续不断的实时遥测数据（自行车被停靠、公交车移动、乘车请求）。你们必须将这两种模式统一到一个单一的“湖仓一体”平台中，以同时支持实时运营和长期城市规划。

必需架构（基本框架）：

- 数据摄取（批处理 + API/模拟流）：**
 - 批处理：** 加载历史参考数据（例如，城市站点位置的CSV文件、人口统计数据或历史天气数据）。
 - 增量数据：** 编写一个脚本，模拟每隔几秒钟到达的实时遥测数据（例如，公交车的GPS信号，或乘车开始/结束事件）。
- 处理（统一引擎）：** 使用PySpark。你们必须将快速移动的事件数据与缓慢变化的历史参考数据连接起来（例如，将实时GPS坐标匹配到静态的区域多边形，或将实时乘车事件与历史天气条件匹配）。
- 存储（严格的分层架构）：**
 - 青铜层：** 原始摄取数据（JSON/CSV）。
 - 白银层：** 清洗、连接后的数据，以 **Delta Lake / Parquet** 格式存储（时间戳标准化，处理空值）。

- **黄金层：**聚合后的业务级表（例如，每个区域的每日出行次数、当前站点利用率）。

4. **服务（分析）：**将简单的仪表板工具（Streamlit、Dash，甚至是带有Matplotlib/Seaborn的Jupyter笔记本）直接连接到你们的黄金层表，以展示洞察结果。

你们的"空白画布"（创意自由）：

- **城市/领域：**纽约CitiBike共享单车数据、芝加哥犯罪数据、伦敦交通数据，或模拟的Uber动态定价数据。
- **分析：**业务价值是什么？你们是在预测哪些自行车站点会在下午5:00前空置？你们是在绘制降雨与交通延误之间的相关性吗？

什么能得'A'：一个完美的分层架构。我希望看到你们的流水线能够轻松处理新一批原始数据，将其通过青铜层、白银层和黄金层进行传播，并立即在最终的仪表板上反映出更新后的聚合结果。

警告：常见失败模式

无论选择哪个方向，都不要落入以下陷阱：

1. **"用户界面陷阱"：**我不关心你们的仪表板是否有漂亮的CSS或时髦的React前端。这是一门数据系统课程。如果你们花20小时在网络用户界面上，而只花2小时在Spark流水线上，你们将不及格。一个能打印出完美工程数据的终端输出，远远优于一个建立在垃圾数据之上的漂亮网络应用。
2. **"上帝脚本"：**不要把所有代码都放在一个巨大的 `main.py` 文件或单个Jupyter Notebook中。要将代码模块化。为数据摄取、处理和服务编写单独的脚本。
3. **"硬编码路径"：**如果你们的代码包含 `filepath = "C:/Users/John/Desktop/data.csv"`，它会在我的机器上崩溃，你们将被扣分。使用相对路径或环境变量。提供 `requirements.txt` 或 `docker-compose.yml`。

明智地选择你们的方向。组建团队。本周就开始构建系统骨架。